Gosling - VolunJá

DOCUMENTO DE ARQUITETURA

Versão 0.3

Tabela 1 – Integrantes do Grupo

Mat.	Nome	Função (responsabilidade)	Pontos de participação na elaboração
231026661	Cauã Henrique Moura Rodrigues	Dev Frontend/Backend	25
232027529	Juan Yuri Gabriel Brito Silva	Scrum Master	25
232005432	Miguel Tobias de Lima Galvão	Dev Frontend/Backend	25
222026528	Rafael Rodrigues Alencar	Dev Frontend/Backend	25
Total de pontos (devem fechar 100 pontos para toda a equipe)			**100**

Fonte: Elaborado pelos autores (2026).

Tabela 2 – Histórico de Revisões

Data	Versão	Descrição	Autor(es)
17/09/2026	0.0	Criação da estrutura inicial dos documentos de visão e arquitetura	Cauã Henrique Moura Rodrigues
05/10/2026	0.1	Elaboração do documento conforme o *template* da disciplina: representação arquitetural, metas e restrições, visões de uso, lógica, estrutural e de implantação, restrições adicionais, referências e padrões de codificação (Apêndice A)	Equipe Gosling
06/10/2026	0.2	Reestruturação conforme o *template* da disciplina: implantação simplificada em contêineres Docker, publicada por túnel da Cloudflare; síntese das tecnologias e dos recursos da API; remoção do glossário, das enumerações e do quadro de ambientes; figuras refeitas no Figma	Equipe Gosling
18/10/2026	0.3	Inclusão de RF26 e RF27: calendário, validação transacional de conflitos, ocorrências de vagas, contrato da API e permissões; roteiro de atualização das figuras no Figma.	Equipe Gosling

Fonte: Elaborado pelos autores (2026).

SUMÁRIO

1 INTRODUÇÃO 3
1.1 PROPÓSITO 3
1.2 ESCOPO 3
2 REPRESENTAÇÃO ARQUITETURAL 5
2.1 DEFINIÇÕES 5
2.2 JUSTIFICATIVA DA ESCOLHA 5
2.3 DETALHAMENTO 7
2.4 METAS E RESTRIÇÕES ARQUITETURAIS 13
2.5 VISÕES 14
2.5.1 Visão de uso 14
2.5.2 Visão de organização lógica 18
2.5.3 Visão estrutural 21
2.6 VISÃO DE IMPLANTAÇÃO 25
2.7 RESTRIÇÕES ADICIONAIS 26
REFERÊNCIAS 29
APÊNDICE A – PADRÕES DE CODIFICAÇÃO E DE VERSIONAMENTO 31
A.1 *BACK-END* (PYTHON) 31
A.2 *FRONT-END* (JAVASCRIPT E REACT) 32
A.3 API REST E BANCO DE DADOS 32
A.4 VERSIONAMENTO 33

1 INTRODUÇÃO

1.1 PROPÓSITO

Este documento descreve a arquitetura do sistema **VolunJá**, desenvolvido pelo grupo Gosling na disciplina de Métodos de Desenvolvimento de Software (MDS), edição do segundo semestre de 2026, a fim de fornecer uma visão abrangente do sistema para desenvolvedores, testadores e demais interessados em aspectos relacionados às tecnologias a serem usadas no desenvolvimento.

O documento registra as decisões arquiteturais da equipe e as justifica a partir do Documento de Visão do Produto e do Projeto. A arquitetura é apresentada por meio de visões complementares — de uso, de organização lógica, estrutural e de implantação —, inspiradas no modelo de visões arquiteturais proposto por Kruchten (1995) e representadas com a Linguagem de Modelagem Unificada — UML (Object Management Group, 2017).

Trata-se de um artefato vivo: ele é atualizado sempre que uma decisão arquitetural for revista ao longo das sprints, e cada revisão é registrada no Histórico de Revisões (Tabela 2). Na apresentação do documento, nas referências e nas citações, seguem-se as normas da Associação Brasileira de Normas Técnicas (2011, 2018, 2023): NBR 14724, NBR 6023 e NBR 10520, respectivamente.

1.2 ESCOPO

O detalhamento do escopo se encontra na Declaração de Escopo do Projeto (seção 4) do Documento de Visão do Produto e do Projeto (Grupo Gosling, 2026), doravante denominado **DV**, que contém os perfis de acesso (Tabela 4), os cenários funcionais (Tabela 5) e o backlog do produto (Tabela 6). Porém, em linhas gerais, o escopo do produto compreende uma aplicação web que conecta voluntários e organizações do terceiro setor, na qual:

a) organizações não governamentais (ONGs) e responsáveis por projetos sociais se cadastram e publicam vagas de voluntariado com campos padronizados: descrição, atividades, área de atuação/causa, modalidade, local, carga horária, período e número de vagas;
b) voluntários se cadastram, mantêm um perfil com interesses e disponibilidade e buscam vagas por filtros sobre esses mesmos campos;

c) o voluntário se candidata a uma vaga, envia a documentação exigida e recebe um comprovante;
d) a organização faz a triagem das candidaturas e, após o aceite, o sistema libera o contato entre o responsável pela vaga e o voluntário;
e) um administrador homologa as organizações e modera vagas e usuários.

o voluntário consulta o calendário das candidaturas PENDENTES e ACEITAS (RF26); novas candidaturas são bloqueadas quando existe sobreposição de horários com outra vaga (RF27).

Ao longo deste documento, as partes do DV usadas nas decisões arquiteturais são referenciadas de forma explícita (por exemplo, “DV, Quadro 5”), e os requisitos são identificados pelos códigos da Tabela 6 do DV (RF01 a RF27 e RNF01 a RNF07).

O documento está organizado em duas seções. Após esta introdução, a seção 2 apresenta a representação arquitetural: as definições, a justificativa da escolha, o detalhamento do estilo, as metas e restrições, as visões de uso, de organização lógica, estrutural e de implantação e as restrições adicionais. Ao final, encontram-se as referências e o Apêndice A, com os padrões de codificação e de versionamento.

Atualização das figuras: os textos e quadros desta revisão incluem RF26 e RF27. As Figuras 1 a 7 ainda são as exportações anteriores do Figma e devem receber as alterações detalhadas em ajustes-figma-calendario.md antes da entrega final das figuras. A Figura 8 não exige mudança de infraestrutura.

2 REPRESENTAÇÃO ARQUITETURAL

2.1 DEFINIÇÕES

Um estilo arquitetural define os tipos de componentes e de conectores de um sistema e as restrições sobre como eles podem ser combinados (Bass; Clements; Kazman, 2021). O sistema seguirá uma **arquitetura cliente-servidor em camadas**, composta por um *front-end* do tipo SPA (*Single Page Application*), desenvolvido em JavaScript com a biblioteca React, e por um *back-end* que expõe uma API (*Application Programming Interface*) no estilo REST (Fielding, 2000), desenvolvido em Python com o *framework* FastAPI. Os dados são armazenados em um banco de dados relacional MySQL.

Na organização em camadas, cada camada oferece serviços à camada superior e depende apenas da camada inferior, o que isola o impacto das mudanças e facilita os testes (Sommerville, 2018). O *back-end* é estruturado como um **monólito modular**: uma única aplicação implantável, dividida internamente em módulos de domínio — autenticação, usuários, vagas, candidaturas, documentos, comprovantes, notificações e moderação —, cada um organizado nas camadas de API, de negócio e de persistência.

2.2 JUSTIFICATIVA DA ESCOLHA

Como recomendam Pressman e Maxim (2016), a escolha do estilo arquitetural partiu da análise de alternativas à luz dos requisitos e das restrições do projeto. Os elementos do DV listados a seguir funcionam como condicionantes da arquitetura:

a) **natureza do produto** (DV, seção 1.1.3 e Quadro 2): o VolunJá é uma aplicação web usada por perfis distintos — voluntários, organizações e administradores — que compartilham um catálogo único de vagas. Centralizar os dados e as regras em um servidor, acessado por clientes leves no navegador, é a forma natural de atender a esse cenário;

b) **foco na experiência do usuário (UX)** (DV, Quadro 1, objetivo específico 6 e requisitos RNF01 a RNF03): uma SPA em React oferece navegação fluida, retorno imediato nos filtros de busca e componentes de interface reutilizáveis, o que favorece a consistência e a acessibilidade. Como o *front-end* é desacoplado, ele pode ser ajustado após cada teste de usabilidade (métrica M6) sem alterar o *back-end*;

c) **tecnologias definidas pela equipe** (DV, Quadro 5): JavaScript com React no *front-end*, Python no *back-end* e MySQL como banco de dados. Entre os *frameworks* Python, o FastAPI foi escolhido por gerar automaticamente a documentação no padrão OpenAPI (OpenAPI Initiative, 2021), que serve de contrato entre *front-end* e *back-end*, por validar os dados de entrada com Pydantic, por oferecer injeção de dependências para autenticação e testes e por apresentar curva de aprendizado reduzida;

d) **equipe, prazo e riscos** (DV, seção 2.1, Tabela 3 e Quadro 8): quatro integrantes e seis sprints de desenvolvimento (Sprints 6 a 11). Um monólito evita a complexidade operacional de sistemas distribuídos, o que mitiga os riscos R2 (inexperiência com as tecnologias) e R3 (escopo grande demais), enquanto a divisão em módulos permite que os integrantes trabalhem em paralelo sobre cenários diferentes;

e) **processo e qualidade** (DV, seções 3.2 e 3.3 e Quadro 11): as práticas do XP (*Extreme Programming*) adotadas, como a integração contínua e os testes automatizados, e a meta de cobertura de testes da métrica M4 (no mínimo 80%) exigem código testável. A separação em camadas, com dependências injetadas, permite testar as regras de negócio isoladamente;

f) **perfis e segurança** (DV, Tabela 4, e requisitos RNF04 e RNF05): o controle de acesso por perfil precisa ser garantido em um único ponto confiável. Na arquitetura cliente-servidor, toda verificação de permissão ocorre no *back-end*, e o *front-end* apenas oculta as opções não permitidas.

O Quadro 1 resume as alternativas de estilo arquitetural avaliadas pela equipe e o motivo da decisão.

Quadro 1 – Alternativas de estilo arquitetural avaliadas

Alternativa	Vantagens	Desvantagens no contexto do VolunJá	Decisão
Monólito com renderização no servidor (padrão MVC, *Model-View-Controller*, com páginas geradas no *back-end*)	Um único projeto e implantação simples.	Acopla interface e regras de negócio; limita a interatividade exigida pelo foco em UX; não aproveita o React definido no DV (Quadro 5).	Descartada
Microsserviços	Escalabilidade e implantação independentes por serviço.	Complexidade de rede, implantação e monitoramento desproporcional a quatro integrantes e seis sprints; agrava os riscos R2 e R3.	Descartada

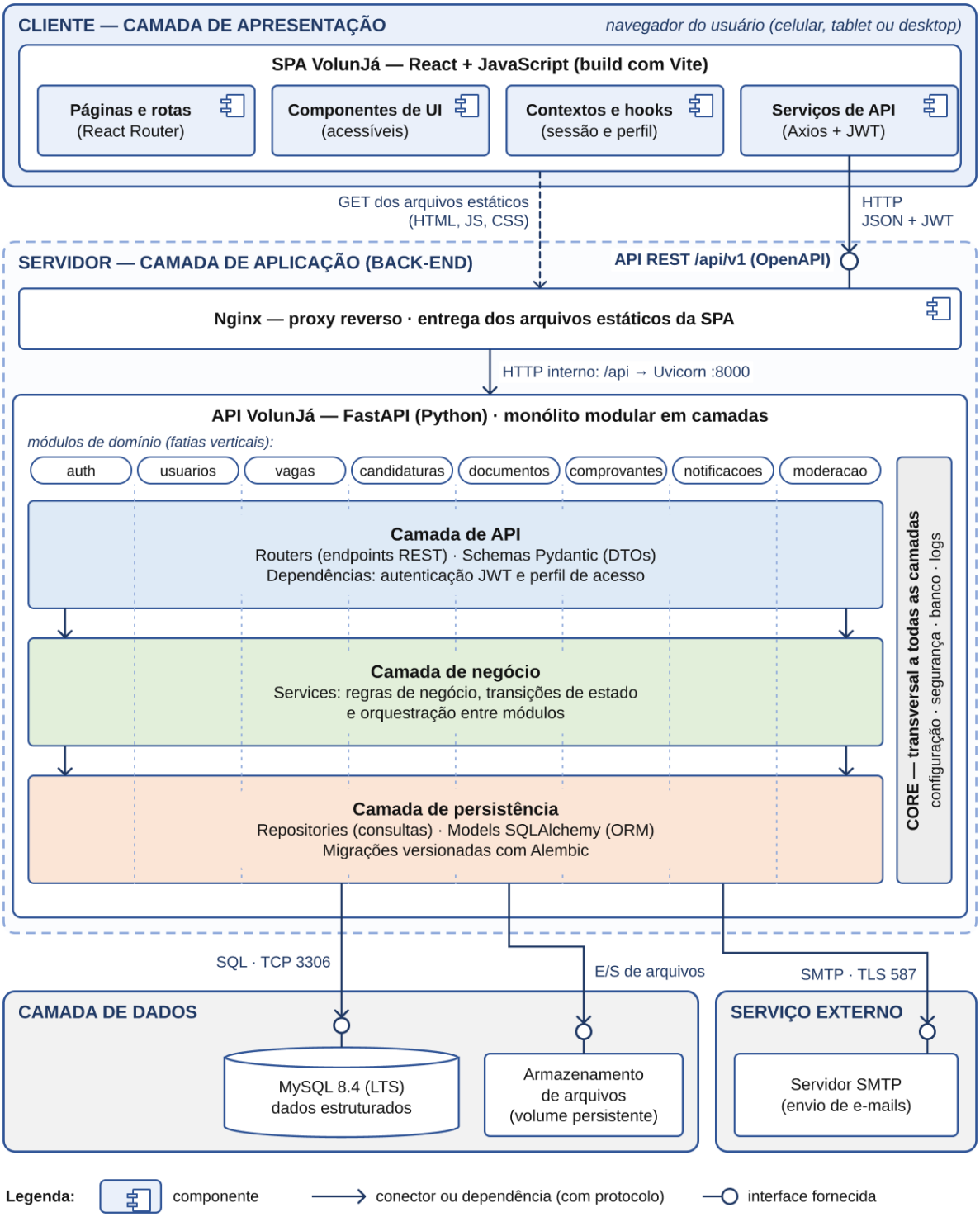
Alternativa	Vantagens	Desvantagens no contexto do VolunJá	Decisão
***Back-end* como serviço (BaaS), por exemplo, Firebase**	Infraestrutura gerenciada e autenticação pronta.	Regras de negócio e de autorização dispersas no cliente; dependência de fornecedor; não utiliza o Python definido no DV.	Descartada
Cliente-servidor em camadas: SPA em React e API REST em FastAPI (monólito modular)	Separação de responsabilidades; trabalho paralelo entre *front-end* e *back-end*; contrato OpenAPI; testabilidade; implantação única.	Duas bases de código (*front-end* e *back-end*) e necessidade de configurar o *proxy* reverso.	**Adotada**

Fonte: Elaborado pelos autores (2026).

2.3 DETALHAMENTO

A Figura 1 representa o estilo arquitetural escolhido. Ela mostra as camadas da arquitetura, os seus componentes principais, as interfaces fornecidas e os conectores, com os respectivos protocolos de comunicação.

Figura 1 – Representação esquemática da arquitetura do VolunJá



Fonte: Elaborada pelos autores.

A arquitetura é composta por cinco camadas — apresentação, API, negócio, persistência e dados —, agrupadas em três blocos na Figura 1. O cliente corresponde à **camada de apresentação**: a SPA, executada no navegador do usuário. O servidor reúne o Nginx, ponto único de entrada, e a API, que contém as camadas de **API**, de **negócio** e de **persistência**. A **camada de dados** é formada pelo MySQL e pelo armazenamento de

arquivos, e o servidor SMTP é um serviço externo, usado para o envio de e-mails. Os módulos de domínio atravessam verticalmente as três camadas internas da API: cada módulo possui os seus próprios *routers*, *services* e *repositories*, e o pacote core oferece serviços transversais a todos eles. Os *services* e os *repositories* seguem, respectivamente, os padrões *Service Layer* e *Repository* descritos por Fowler (2002).

Os elementos do estilo arquitetural são instanciados com as tecnologias do Quadro 2, selecionadas a partir das suas documentações oficiais (React, 2026; FastAPI, 2026; SQLAlchemy, 2026; MySQL, 2026; Docker, 2026).

Quadro 2 – Tecnologias adotadas por elemento da arquitetura

Elemento	Tecnologia	Justificativa
SPA (apresentação)	JavaScript (ES2022+), React 19, Vite e React Router	React definido no DV (Quadro 5); o Vite gera um *build* otimizado e oferece recarga rápida no desenvolvimento; o React Router organiza as telas por rota e por perfil.
Servidor web e *proxy* reverso	Nginx	Entrega os arquivos estáticos com compressão e *cache* e encaminha /api para a aplicação, o que mantém a SPA e a API na mesma origem e dispensa a configuração de compartilhamento de recursos entre origens (CORS).
Camada de API	FastAPI, Pydantic v2 e Uvicorn (ASGI)	Validação declarativa da entrada, documentação OpenAPI automática e injeção de dependências para autenticação e autorização.
Camada de negócio	Classes de serviço em Python 3.12	Regras de negócio independentes do HTTP e do banco de dados, testáveis isoladamente.
Camada de persistência	SQLAlchemy 2.0, Alembic e PyMySQL	Mapeamento objeto-relacional (ORM) maduro, com suporte ao MySQL; migrações do banco versionadas junto com o código.
Banco de dados	MySQL 8.4 (LTS)	Definido no DV (Quadro 5); relacional, adequado a entidades fortemente relacionadas que exigem integridade referencial e restrições de unicidade.
Armazenamento de arquivos	Volume persistente no servidor	Solução simples para os documentos das candidaturas, acessada apenas pelo módulo de documentos e substituível por armazenamento de objetos sem impacto nas demais camadas.
Segurança	JWT (*JSON Web Token*), com PyJWT, e *hash* de senhas com Argon2 (pwdlib)	Autenticação sem estado no servidor e armazenamento seguro das senhas (RNF04).
E-mail	SMTP e tarefas em segundo plano do FastAPI (BackgroundTasks)	Envio de e-mails sem aumentar o tempo de resposta das requisições (RF05 e RF20).
Contêineres	Docker e Docker Compose	Mesma composição de serviços em qualquer máquina da equipe.
Integração contínua	GitHub Actions	Já utilizado na publicação da documentação (DV, Quadro 5); executa *lint* e testes a cada *pull request*.

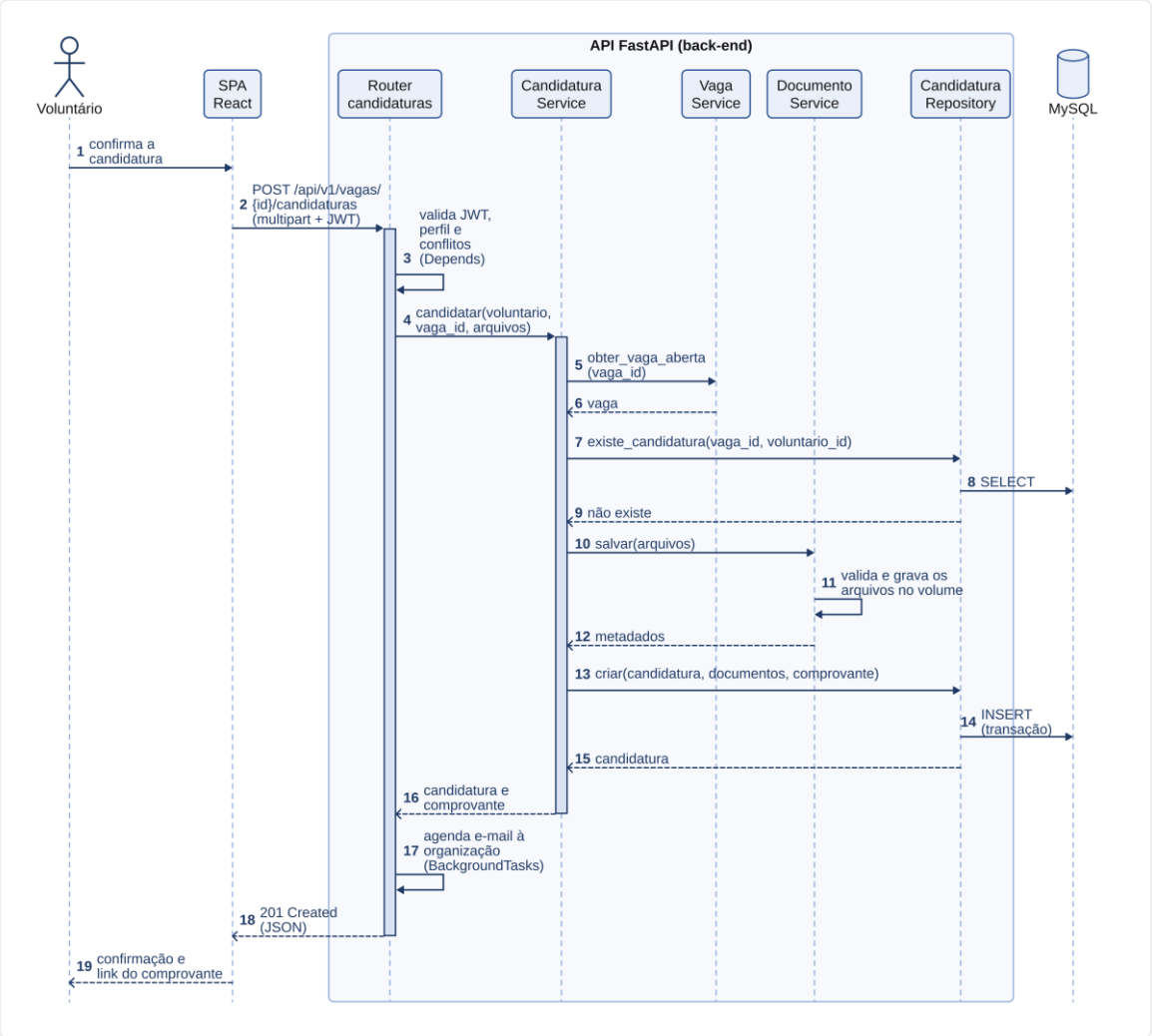
Fonte: Elaborado pelos autores (2026).

Completam a pilha as bibliotecas de apoio — Axios, no cliente HTTP; CSS Modules, na estilização; e ReportLab, na geração do comprovante em PDF (RF22) — e as ferramentas de qualidade e testes previstas no DV (seção 6.1).

A Figura 2 mostra os elementos da Figura 1 em funcionamento, por meio do diagrama de sequência da candidatura a uma vaga (RF16 e RF17). A requisição percorre as camadas de cima para baixo, e a resposta retorna no sentido inverso; o e-mail de aviso à organização é enviado em segundo plano, depois que a resposta já foi devolvida ao voluntário.

Para RF27, o fluxo passa a validar os horários antes de salvar arquivos, candidatura e comprovante. O serviço abre uma transação, bloqueia a linha do voluntário, consulta as candidaturas ativas e compara todas as ocorrências. Com conflito, desfaz a transação e devolve HTTP 409 com os dados da vaga conflitante; sem conflito, persiste e confirma a transação antes de notificar.

Figura 2 – Interação entre as camadas na candidatura a uma vaga (diagrama de sequência)



Fonte: Elaborada pelos autores.

O Quadro 3 explica a responsabilidade de cada elemento da arquitetura e as suas regras de uso, isto é, as entradas esperadas, as saídas produzidas e as condições para que a interface funcione adequadamente.

Quadro 3 – Responsabilidades e regras de uso dos elementos da arquitetura

Elemento	Responsabilidade	Entradas esperadas	Saídas e regras de uso
SPA (React)	Exibir a interface, coletar e validar previamente os dados e controlar a navegação por perfil.	Ações do usuário; respostas JSON da API.	Requisições HTTP com JSON ou arquivos (*multipart/form-data*) e *token* JWT. Não contém regras de negócio: a validação no cliente serve apenas para dar retorno imediato ao usuário.
Serviços de API (*front-end*)	Centralizar o acesso à API REST.	Parâmetros vindos das telas.	Chamadas HTTP com o cabeçalho Authorization; diante da resposta 401, encerram a sessão e direcionam ao *login*.

Elemento	Responsabilidade	Entradas esperadas	Saídas e regras de uso
Nginx	Ser o ponto único de entrada da aplicação.	Requisições HTTP na porta 80.	Arquivos estáticos da SPA; encaminhamento de /api ao Uvicorn (porta 8000); cabeçalhos de segurança e limite de tentativas de *login*.
Routers	Expor os *endpoints* REST versionados (/api/v1) e traduzir o HTTP em chamadas de serviço.	Requisições validadas pelos *schemas*.	Respostas JSON com códigos HTTP padronizados (200, 201, 204, 400, 401, 403, 404, 409 e 422). Não acessam o banco de dados diretamente.
***Schemas* (Pydantic)**	Definir os contratos de entrada e de saída (DTOs, objetos de transferência de dados).	Dados brutos da requisição.	Objetos validados ou erro 422. Nunca expõem campos sensíveis, como o *hash* da senha.
Dependências	Autenticar o usuário, verificar o perfil e abrir a sessão de banco de dados por requisição.	*Token* JWT; perfis exigidos pelo *endpoint*.	Usuário autenticado ou erro 401/403; sessão de banco de dados encerrada ao fim da requisição.
Services	Aplicar as regras de negócio, controlar as transações e orquestrar os módulos.	Objetos validados e usuário atual.	Entidades e resultados, ou exceções de negócio convertidas em respostas HTTP por um tratador central. Não dependem do HTTP.
Repositories	Encapsular as consultas e a persistência.	Critérios de consulta e entidades.	Entidades do ORM. São os únicos elementos que usam a sessão do SQLAlchemy.
***Models* (ORM)**	Mapear tabelas e relacionamentos e implementar regras de estado simples, como aceitar uma candidatura.	Dados persistidos.	Objetos de domínio usados pelos *services*.
Core	Fornecer configuração, segurança, sessão de banco de dados, tratamento de erros e logs.	Variáveis de ambiente.	Configurações tipadas e funções utilitárias usadas por todos os módulos.
MySQL	Persistir os dados estruturados com integridade referencial.	Comandos SQL gerados pelo SQLAlchemy.	Resultados das consultas; chaves estrangeiras e restrições de unicidade garantem a consistência, como uma única candidatura por voluntário e vaga.
Armazenamento de arquivos	Guardar os documentos das candidaturas.	Arquivos já validados (PDF, JPG ou PNG, até 5 MB).	Caminho interno do arquivo; acesso somente pela API, após a verificação de permissão (RNF04).
Servidor SMTP	Entregar os e-mails do sistema.	Mensagens do módulo de notificações.	E-mails de recuperação de senha (RF05) e de resultado da candidatura (RF20).

Fonte: Elaborado pelos autores (2026).

Além das responsabilidades, quatro regras de dependência garantem a integridade do estilo:

a) cada camada depende apenas da camada imediatamente inferior (*router*, *service*, *repository* e model), nunca no sentido inverso;
b) um módulo acessa outro somente por meio do *service* desse módulo, e nunca diretamente pelo *repository* ou pelos *models*;
c) o *front-end* comunica-se com o *back-end* exclusivamente pela API REST documentada em OpenAPI;

d) toda regra de negócio e toda verificação de permissão ficam no *back-end*.

2.4 METAS E RESTRIÇÕES ARQUITETURAIS

O Quadro 4 apresenta as metas e as restrições de funcionamento que orientam o projeto e a implementação. Cada item recebe um identificador (MR) para que possa ser rastreado nos critérios de aceitação e nos testes (DV, seção 6).

Quadro 4 – Metas e restrições arquiteturais

ID	Meta ou restrição	Justificativa	Verificação
MR01	O *back-end* deve ser desenvolvido em Python 3.12 ou superior com FastAPI; o *front-end*, em JavaScript com React; e o banco de dados deve ser o MySQL 8.4.	Tecnologias definidas pela equipe (DV, Quadro 5) e conhecidas pelos integrantes, em coerência com a mitigação do risco R2 (DV, Quadro 8).	Revisão de código; dependências declaradas nos arquivos do projeto.
MR02	A API deve seguir o estilo REST, com JSON, prefixo de versão /api/v1 e documentação OpenAPI gerada automaticamente.	Contrato único entre *front-end* e *back-end*, que permite desenvolvê-los em paralelo.	Documentação disponível em /docs; testes de integração.
MR03	95% das buscas de vagas com filtros (RF13) devem responder em até 2 segundos, com até 10.000 vagas cadastradas; as listagens são paginadas (20 itens por página).	Desempenho da busca (RNF06): a busca é a principal tarefa do voluntário (métrica M6).	Tempo de resposta registrado pela API em testes de sistema com massa de dados.
MR04	O código deve seguir os padrões de codificação do Apêndice A.	Manutenibilidade (RNF07): o DV remete os padrões de código a este documento.	*Lint* automático na integração contínua; *pull request* bloqueado em caso de falha.
MR05	A cobertura de testes do *back-end* deve ser de, no mínimo, 80%.	Métrica M4 (DV, Quadro 11) e conceito de pronto (DV, seção 3.3).	Relatório do pytest-cov a cada *pull request*.
MR06	Toda operação que altera dados ou acessa dados pessoais de usuário logado deve exigir autenticação por JWT e autorização por perfil e por propriedade do recurso.	Segurança dos dados e controle de acesso por perfil (RNF04 e RNF05; DV, Tabela 4).	Caso de teste CT06 (DV, Tabela 7) e testes de integração por perfil.
MR07	A interface deve ser responsiva a partir de 360 px de largura e atender às diretrizes WCAG 2.1 no nível AA.	Responsividade e acessibilidade (RNF02 e RNF03) e foco em UX (DV, Quadro 1).	Testes com Playwright em telas de celular, tablet e desktop; verificação com axe-core sem violações graves.
MR08	A aplicação deve ser executável em contêineres Docker, com a mesma composição em qualquer máquina da equipe.	Evita diferenças de configuração entre as máquinas dos integrantes e simplifica a preparação do ambiente na Sprint 5 (DV, Tabela 3).	O comando docker compose up inicia o sistema completo.
MR09	Os documentos das candidaturas devem ser arquivos PDF, JPG ou PNG de até 5 MB cada.	Segurança dos dados (RNF04): reduz o risco de arquivos maliciosos e o uso excessivo de disco.	Testes de envio com tipos e tamanhos inválidos.
MR10	A verificação de conflitos e o registro da candidatura devem ser atômicos por voluntário, inclusive sob requisições simultâneas.	RF27 e RN01 a RN05 do DV.	Testes CT09 a CT12 e CT14 do DV; apenas uma candidatura sobreposta pode ser confirmada.

Fonte: Elaborado pelos autores (2026).

Também são de interesse dos desenvolvedores a política de *branches*, a convenção de mensagens de *commit* e as regras de revisão de código, descritas no Apêndice A, junto com os padrões de codificação.

2.5 VISÕES

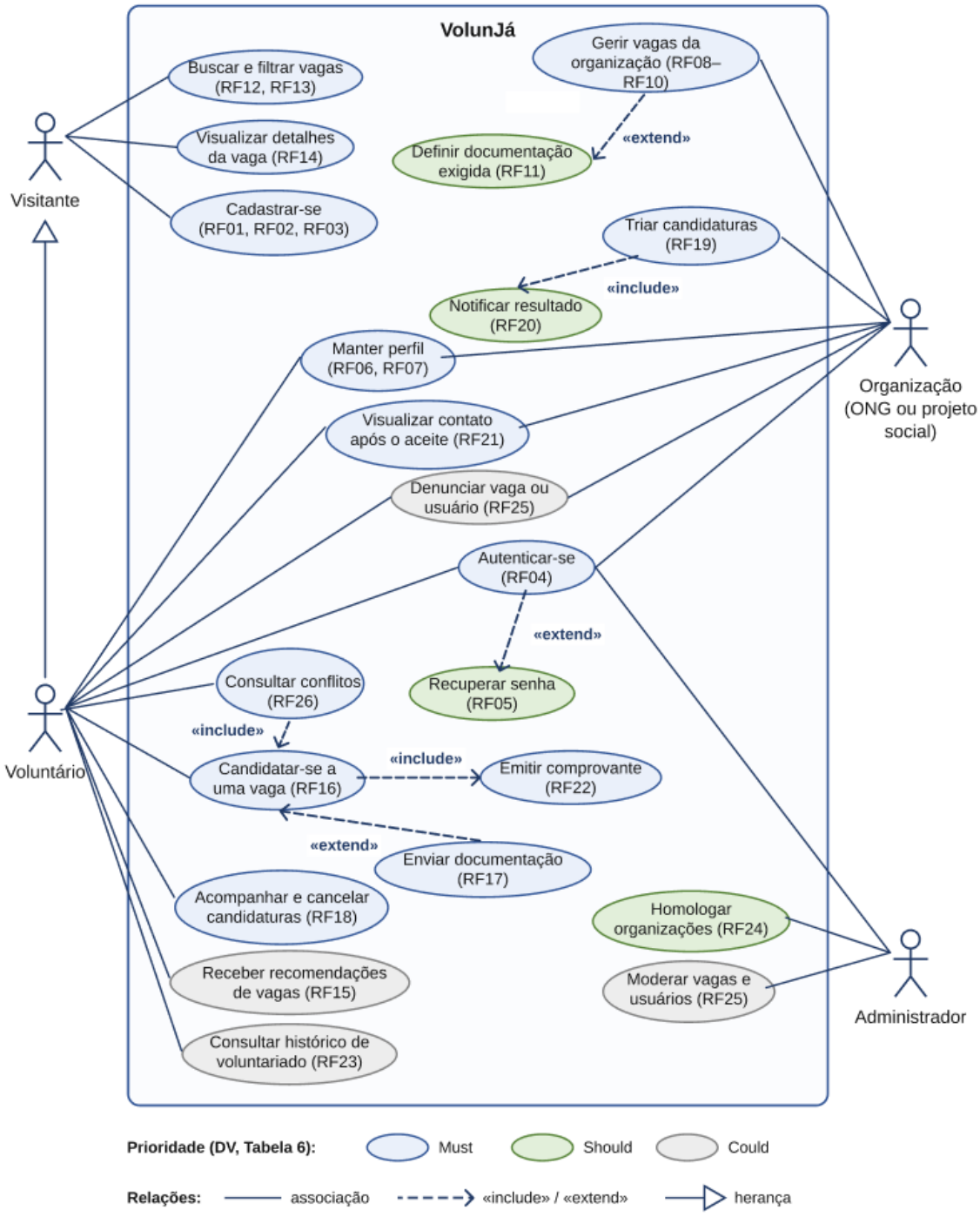
A arquitetura é comunicada por quatro visões complementares:

a) **visão de uso** (seção 2.5.1): o escopo do sistema, por meio dos casos de uso e da funcionalidade mais complexa;
b) **visão de organização lógica** (seção 2.5.2): os módulos, as camadas e as formas de comunicação entre eles;
c) **visão estrutural** (seção 2.5.3): a composição do sistema em classes e componentes e as relações entre as suas partes;
d) **visão de implantação** (seção 2.6): o ambiente em que o sistema será executado.

2.5.1 Visão de uso

Em poucas palavras, o VolunJá permite que organizações publiquem vagas de voluntariado padronizadas e que voluntários as encontrem por compatibilidade, candidatem-se com a documentação exigida, recebam um comprovante e sejam conectados ao responsável pela vaga após o aceite. A Figura 3 apresenta o diagrama de casos de uso: os atores correspondem aos perfis da Tabela 4 do DV, cada caso de uso indica os requisitos da Tabela 6 do DV que realiza, e as cores indicam a prioridade MoSCoW.

Figura 3 – Diagrama de casos de uso do VolunJá



Fonte: Elaborada pelos autores.

O Voluntário herda os casos de uso do Visitante, pois a Tabela 4 do DV atribui a ele todas as permissões desse perfil. Os casos de uso compartilhados por voluntários e organizações — manter perfil, visualizar contato, denunciar e autenticar-se — ficam no centro do diagrama. As relações «extend» representam comportamentos opcionais, como o envio de

documentação, que ocorre apenas quando a vaga a exige, e as relações «include» representam comportamentos sempre executados, como a emissão do comprovante em toda candidatura.

Alguns requisitos do DV contribuíram diretamente para a escolha do estilo arquitetural e para a organização interna do sistema, conforme o Quadro 5.

Quadro 5 – Requisitos com impacto na arquitetura

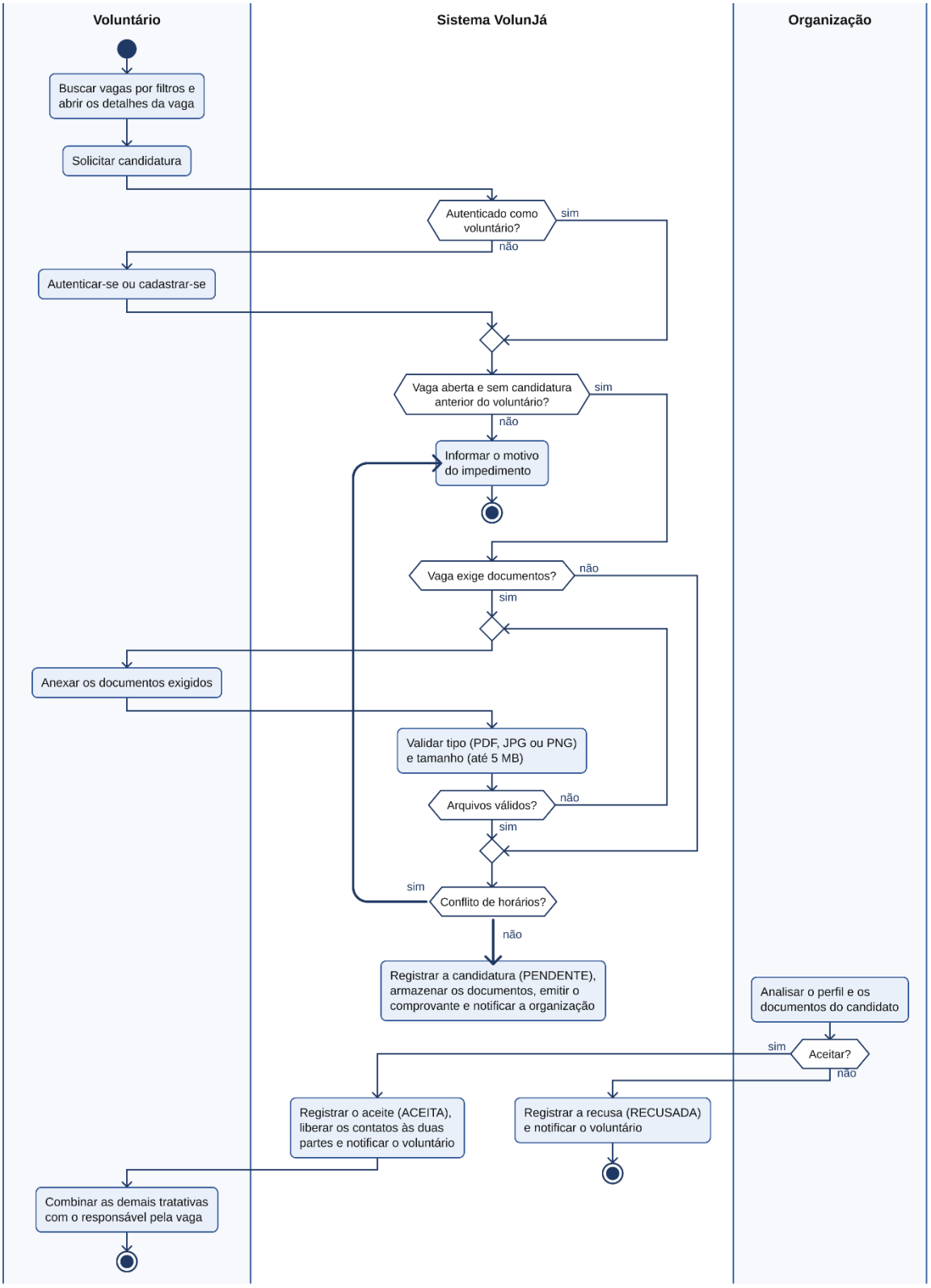
Requisitos (DV, Tabela 6)	Impacto na arquitetura
RNF04 e RNF05 (segurança dos dados e controle de acesso por perfil)	Autorização centralizada no *back-end*, por meio de dependências do FastAPI, e verificação de propriedade nos *services*; documentos entregues apenas pela API, nunca como arquivos públicos.
RNF01, RNF02 e RNF03 (usabilidade, responsividade e acessibilidade)	*Front-end* desacoplado (SPA), com componentes reutilizáveis e acessíveis, que evolui a cada teste de usabilidade sem alterar o *back-end*.
RF13 e RNF06 (busca por filtros e desempenho)	Campos de filtro armazenados como colunas indexadas no MySQL; consultas paginadas montadas no *repository* de vagas.
RF17 e RF22 (documentação e comprovante)	Módulos próprios de documentos e de comprovantes; arquivos guardados fora do banco de dados, em volume persistente.
RF05 e RF20 (e-mails)	Envio assíncrono pelo módulo de notificações, para não aumentar o tempo de resposta das requisições.
RF24 e RF25 (homologação e moderação)	Módulo de moderação isolado, acessível apenas ao perfil Administrador.
RNF07 (manutenibilidade)	Monólito modular em camadas e padrões de codificação definidos no Apêndice A.
RF26 e RF27 (calendário e conflito de horário)	Agenda derivada das candidaturas ativas e das ocorrências das vagas; validação no serviço de candidaturas, com transação e bloqueio por voluntário.

Fonte: Elaborado pelos autores (2026).

Além dos requisitos, pesaram na escolha a decisão da equipe de adotar tecnologias já conhecidas pelos integrantes — Python, JavaScript e React —, registrada como mitigação do risco R2 (DV, Quadro 8); a documentação extensa do FastAPI e do React, que reduz o tempo de aprendizado; e o prazo de seis sprints de desenvolvimento (DV, Tabela 3), que desaconselha estilos distribuídos.

A funcionalidade mais complexa é a **candidatura a uma vaga** (cenário CN06), com o seu desdobramento na triagem e na conexão entre as partes (CN07 e CN08). Ela envolve três participantes, envio e validação de arquivos, regras de estado da candidatura, emissão de comprovante, notificações e liberação controlada de dados de contato. A Figura 4 apresenta o seu diagrama de atividades.

Figura 4 – Diagrama de atividades da candidatura a uma vaga



Fonte: Elaborada pelos autores.

O fluxo da Figura 4 evidencia as regras que a camada de negócio deve garantir: somente voluntários autenticados podem se candidatar; a vaga precisa estar aberta; cada

voluntário pode ter apenas uma candidatura por vaga; os arquivos são validados antes do registro; e os contatos só são liberados após o aceite. Enquanto a candidatura estiver pendente, o voluntário pode cancelá-la (RF18).

Também é obrigatório validar ausência de sobreposição com candidaturas PENDENTES ou ACEITAS (RF27). A interface mostra a vaga conflitante e oferece o acesso à agenda (RF26); o bloqueio não depende de JavaScript do navegador. RECUSADAS e CANCELADAS liberam o horário. ENCERRADA indica o fim das inscrições da vaga e, isoladamente, não libera compromissos ativos.

2.5.2 Visão de organização lógica

O sistema é subdividido em dois subsistemas — o *front-end* (SPA) e o *back-end* (API) — e cada um deles, em módulos com responsabilidades próprias. O Quadro 6 lista os módulos, a razão lógica de cada um, os requisitos que atendem e as suas dependências.

Quadro 6 – Módulos do sistema

Módulo	Razão lógica	Requisitos (DV)	Depende de
***Front-end* (SPA em React)**			
app	Ponto de entrada: rotas, layout e provedores de contexto; restringe as rotas de acordo com o perfil.	RF04, RNF05	pages; contexts e *hooks*
pages	Telas por perfil, incluindo calendário e agenda diária do voluntário, e mensagem de conflito na candidatura.	RF01 a RF27	components; contexts e hooks; services
components	Componentes de interface reutilizáveis e acessíveis, como formulários, cartões de vaga, filtros e janelas modais.	RNF01 a RNF03	utils e styles
contexts e *hooks*	Estado compartilhado (sessão e perfil) e lógica reutilizável entre as telas.	RF04	*services*
services	Acesso à API REST, com um serviço por recurso e um cliente HTTP configurado.	Todos	API REST (*back-end*)
utils e styles	Validações (CPF e CNPJ), formatação de dados e tema visual.	RNF02	—
***Back-end* (API em FastAPI)**			
core	Configuração, segurança (*hash* e JWT), sessão de banco de dados, erros e logs.	RNF04	—
auth	*Login*, emissão de *token* e recuperação de senha.	RF04, RF05	usuarios; notificacoes
usuarios	Contas, voluntários, organizações e perfis.	RF01 a RF03, RF06, RF07	—
vagas	Publicação, edição, busca e ocorrências com início e fim; horários imutáveis enquanto houver candidatura ativa.	RF08 a RF15; apoio a RF26 e RF27	usuarios
candidaturas	Candidatura, agenda do voluntário, validação de conflitos, acompanhamento, cancelamento, triagem e contato.	RF16, RF18, RF19, RF21, RF26, RF27	usuarios; vagas; documentos; comprovantes; notificacoes

Módulo	Razão lógica	Requisitos (DV)	Depende de
documentos	Recebimento, validação, armazenamento e acesso controlado aos arquivos.	RF17, RNF04	—
comprovantes	Emissão do comprovante em PDF e histórico de voluntariado.	RF22, RF23	—
notificacoes	Notificações internas e envio de e-mails em segundo plano.	RF05, RF20	—
moderacao	Homologação de organizações, denúncias e suspensão de contas.	RF24, RF25	usuarios; vagas; notificacoes

Fonte: Elaborado pelos autores (2026).
Nota: todos os módulos do *back-end* utilizam o core, que por isso não é repetido na coluna "Depende de".

Os módulos comunicam-se de três formas:

a) **entre *front-end* e *back-end***: requisições HTTP com JSON pela API REST, com o *token* JWT no cabeçalho Authorization; os documentos são enviados como *multipart/form-data* e os comprovantes são recebidos em PDF. O Quadro 7 lista os principais recursos da API;

b) **entre módulos do *back-end***: chamadas de função dentro do mesmo processo, sempre de um *service* para outro, respeitando as regras de dependência da seção 2.3;

c) **entre o *back-end* e a infraestrutura**: SQL sobre TCP (porta 3306) com o MySQL, leitura e gravação de arquivos no volume persistente e SMTP com TLS (porta 587) com o servidor de e-mail.

Quadro 7 – Principais recursos da API REST por módulo

Módulo	Principais recursos	Perfis	Requisitos
auth	POST /auth/login; POST /auth/recuperacao-senha	Público	RF04, RF05
usuarios	POST /voluntarios; POST /organizacoes; GET e PUT /voluntarios/me e /organizacoes/me	Público (cadastro); Voluntário; Organização	RF01 a RF03, RF06, RF07
vagas	GET /vagas; GET /vagas/{id}; GET /vagas/recomendadas; POST /vagas; PUT e PATCH /vagas/{id}; GET /organizacoes/me/vagas	Público (consulta); Voluntário; Organização dona	RF08 a RF15
candidaturas	POST /vagas/{id}/candidaturas; GET /voluntarios/me/candidaturas; GET /voluntarios/me/calendario; GET /vagas/{id}/candidaturas; PATCH /candidaturas/{id}; GET /candidaturas/{id}/contato	Voluntário; Organização dona; Envolvidos	RF16 a RF21, RF26, RF27
documentos	GET /documentos/{id}	Envolvidos	RF17, RNF04
comprovantes	GET /candidaturas/{id}/comprovante	Voluntário	RF22
moderacao	PATCH /admin/organizacoes/{id}; GET e PATCH /admin/denuncias; POST /denuncias	Administrador; Autenticado (denúncia)	RF24, RF25

Fonte: Elaborado pelos autores (2026).
Nota: todos os caminhos têm o prefixo /api/v1. "Envolvidos" são o voluntário da candidatura e a organização dona da vaga, conforme a ação permitida a cada um (Quadro 9). A documentação completa é gerada automaticamente pelo FastAPI, no padrão OpenAPI, e fica disponível em /docs.

Contrato do calendário (RF26): GET /api/v1/voluntarios/me/calendario?inicio=2026-11-16&fim=2026-11-23 consulta um intervalo de datas com início inclusivo e fim exclusivo. Exige JWT de voluntário e usa a identidade do token. Retorna HTTP 200 com eventos contendo candidatura_id, vaga_id, titulo_vaga, organizacao, status, inicio e fim; sem eventos, retorna lista vazia. As datas delimitam dias no fuso America/Sao_Paulo; parâmetros ausentes, inválidos ou inicio >= fim retornam 422.

Contrato de conflito (RF27): POST /api/v1/vagas/{id}/candidaturas retorna HTTP 409 com codigo=CONFLITO_HORARIO, mensagem e conflitos contendo vaga_id, titulo_vaga, inicio e fim de cada ocorrência conflitante da própria agenda. A SPA apresenta os nomes e os horários e a ação Ver meu calendário. O retorno 201 ocorre somente após a confirmação da transação; não se emite comprovante nem aviso de sucesso no caminho de conflito.

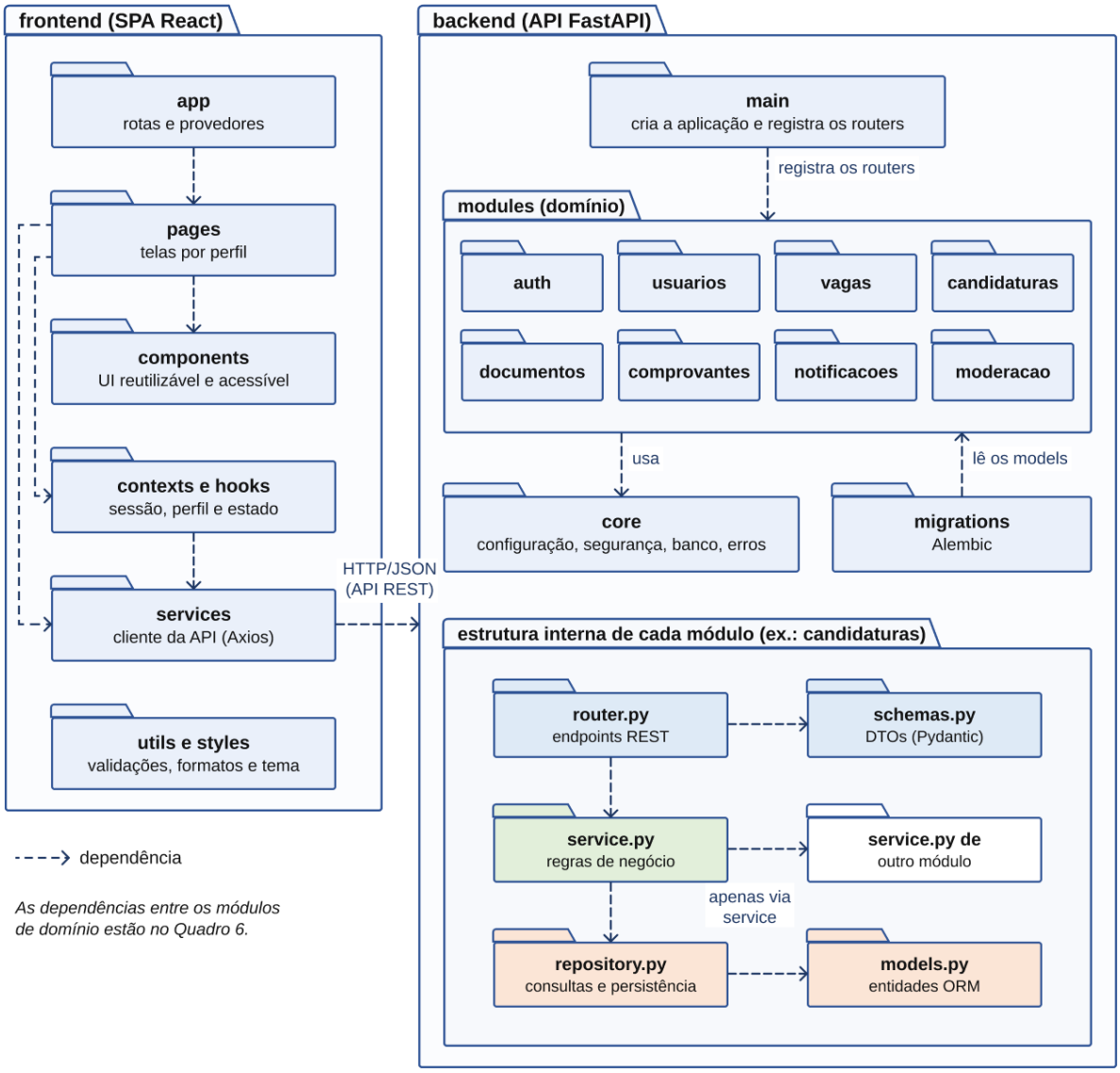
Concorrência e persistência: o CandidaturaService controla a transação. Por meio do UsuarioService e de seu repository, obtém bloqueio SELECT FOR UPDATE na linha do voluntário, inclusive quando ele ainda não possui candidaturas. Após adquirir o bloqueio, usa leituras atuais com bloqueio para consultar as candidaturas ativas e suas ocorrências; valida e grava na mesma transação. Criar, cancelar, aceitar ou recusar candidaturas deve respeitar a mesma disciplina de bloqueio. A restrição UNIQUE(voluntario_id, vaga_id) continua necessária, mas sozinha não impede sobreposições entre vagas diferentes.

Edição e ordem dos bloqueios: criar candidatura e editar horários também bloqueiam a linha da vaga antes de ler seus horários ou verificar candidaturas ativas. Quando uma operação precisa dos dois bloqueios, a ordem é voluntário e depois vaga; alterações que atinjam vários voluntários ordenam os IDs. O serviço de vagas recusa alteração de horários se houver candidatura PENDENTE ou ACEITA, evitando invalidar a agenda já assumida.

Comparação temporal: armazenar início e fim em UTC e converter para America/Sao_Paulo na interface. Há conflito se inicio_novo < fim_existente e fim_novo > inicio_existente. Comparar todas as ocorrências, inclusive intervalos entre dias; horários consecutivos são permitidos. Índices em candidaturas(voluntario_id, status) e ocorrencias_vaga(vaga_id, inicio, fim) apoiam as consultas. A agenda é uma projeção dos dados existentes, sem tabela de calendário duplicada.

A Figura 5 apresenta o diagrama de pacotes, com a organização do código dos dois subsistemas e a estrutura interna padrão dos módulos do *back-end*.

Figura 5 – Diagrama de pacotes do VolunJá



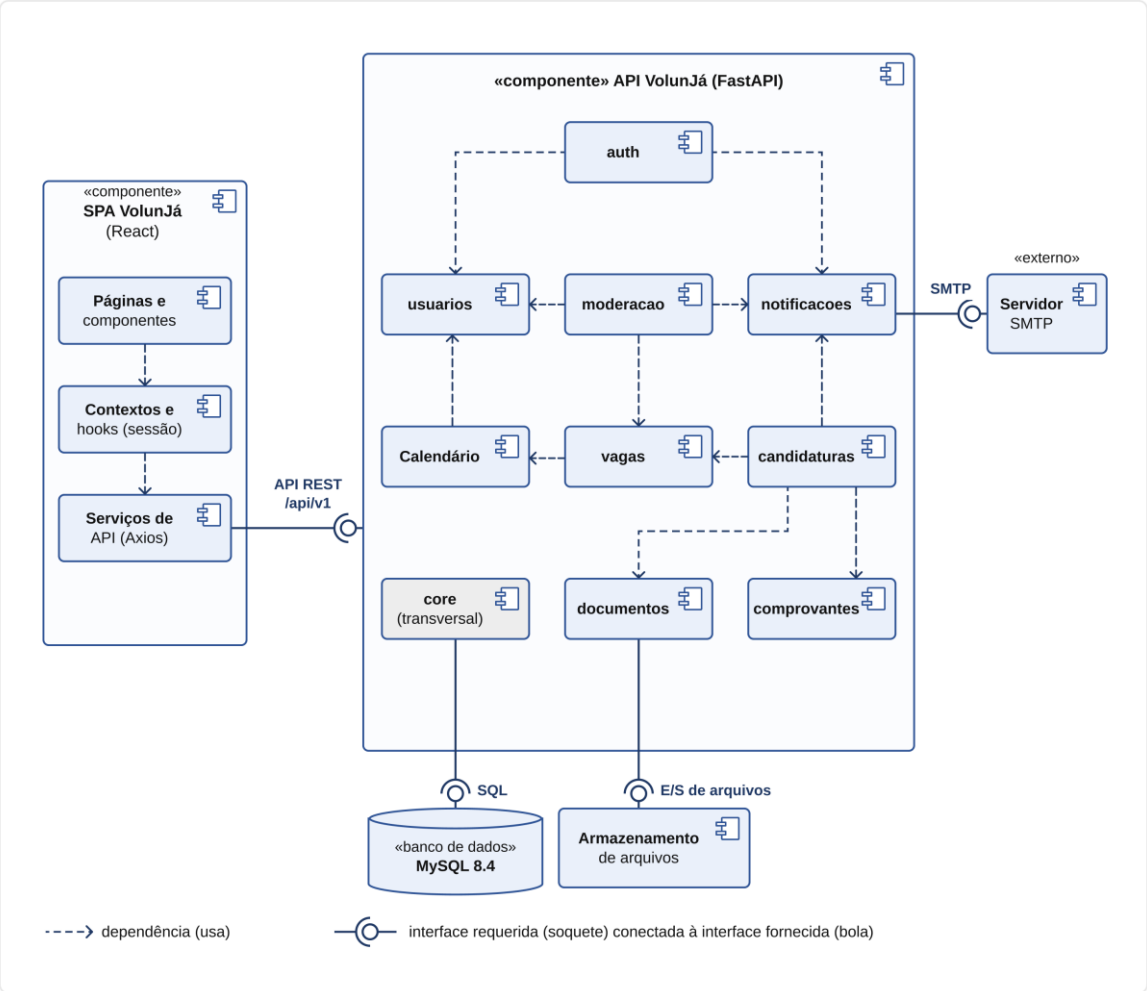
Fonte: Elaborada pelos autores.

Todos os módulos do *back-end* seguem a mesma estrutura de arquivos: router.py (*endpoints*), schemas.py (DTOs), service.py (regras de negócio), repository.py (consultas) e models.py (entidades). Essa padronização reduz o tempo de aprendizado ao mudar de módulo, o que favorece a prática de código coletivo adotada no processo (DV, seção 3.2).

2.5.3 Visão estrutural

A visão estrutural é apresentada em dois níveis: os componentes de execução, no diagrama de componentes (Figura 6), e o modelo de domínio, no diagrama de classes (Figura 7).

Figura 6 – Diagrama de componentes do VolunJá



Fonte: Elaborada pelos autores.

O diagrama de componentes da Figura 6 mostra como as partes do sistema se conectam por meio de interfaces. A SPA requer a interface API REST, fornecida pela API VolunJá; dentro da API, os módulos dependem uns dos outros conforme o Quadro 6, sempre por meio dos seus *services*; e a API requer as interfaces SQL, de arquivos e SMTP, fornecidas, respectivamente, pelo MySQL, pelo armazenamento de arquivos e pelo servidor de e-mail.

Nenhum módulo depende de auth, candidaturas ou moderacao, que ficam no topo da cadeia de dependências, enquanto usuarios, documentos, comprovantes e notificacoes são módulos de base, que não dependem de outros módulos de domínio. Essa organização, sem dependências circulares, permite testar cada módulo isoladamente, substituindo as suas dependências por dublês de teste.

A Figura 7 define as entidades persistidas e os seus relacionamentos. As classes têm as seguintes responsabilidades:

a) **Usuario** concentra os dados de acesso comuns a todos os perfis. **Voluntario** e **Organizacao** especializam Usuario de forma disjunta, refletindo a regra de que os perfis de voluntário e de organização são exclusivos em uma mesma conta (DV, Tabela 4); o administrador é um Usuario com o perfil ADMINISTRADOR;

b) **Organizacao** representa tanto ONGs quanto responsáveis por projetos sociais (atributo tipo), identificados por CNPJ ou CPF (RF02 e RF03), e possui um estado de homologação (RF24);

c) **Vaga** reúne exatamente os campos padronizados do DV, que são também os critérios de filtro da busca (RF08 e RF13); os seus métodos controlam o encerramento e informam se a vaga ainda aceita candidaturas;

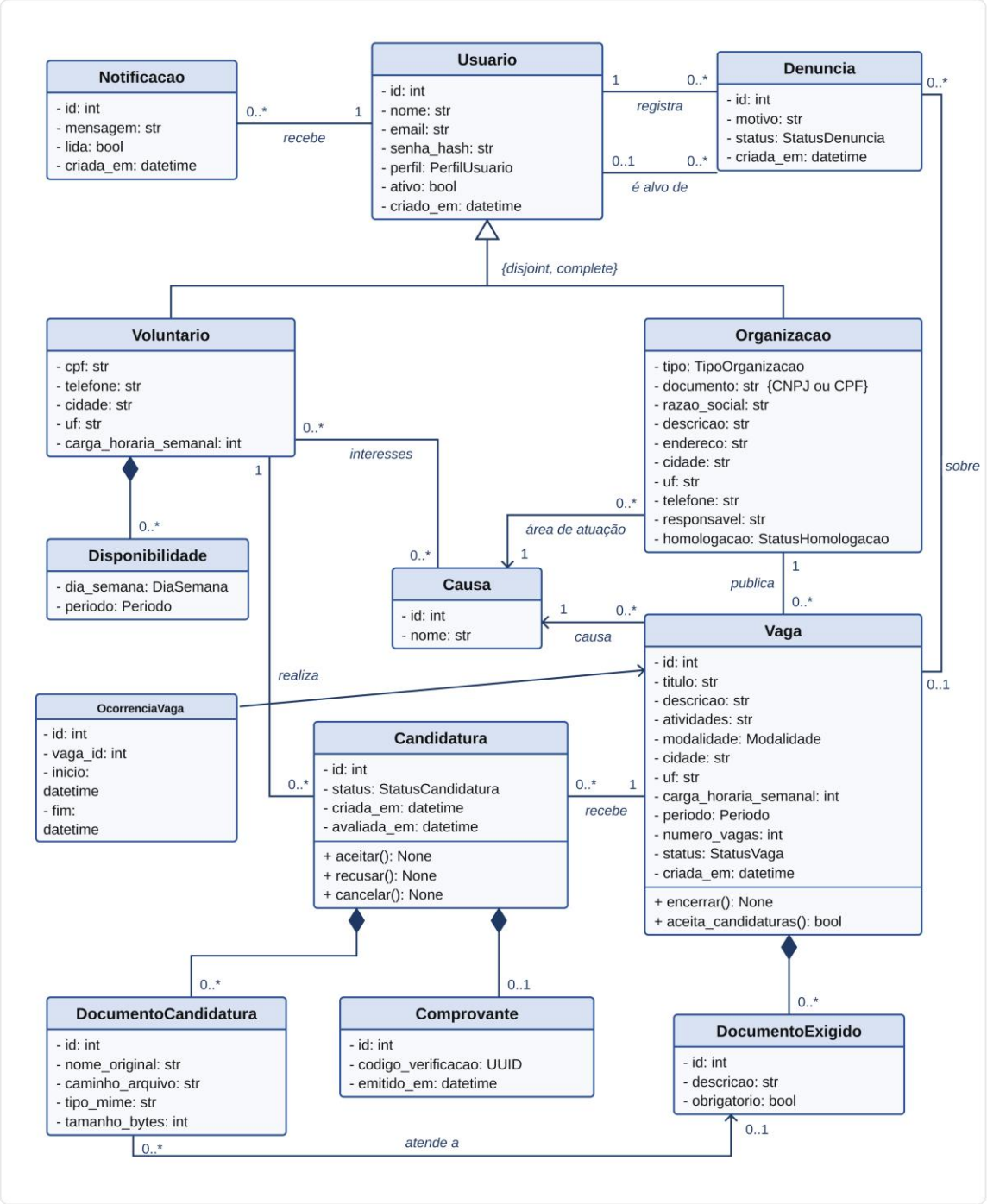
d) **Causa** é um catálogo compartilhado pelas vagas, pelas áreas de atuação das organizações e pelos interesses dos voluntários; com a **Disponibilidade**, permite calcular a compatibilidade entre voluntários e vagas (RF15);

e) **Candidatura** liga um voluntário a uma vaga, com combinação única, e controla as transições de estado: de PENDENTE para ACEITA, RECUSADA ou CANCELADA. Ela é composta pelos **DocumentoCandidatura** enviados, cada um vinculado ao **DocumentoExigido** que atende, e pelo **Comprovante**, que possui um código de verificação;

f) **Notificacao** registra os avisos internos enviados aos usuários (RF20), e **Denuncia** registra as denúncias sobre vagas ou usuários, tratadas pelo administrador (RF25).

7) OcorrenciaVaga representa cada intervalo agendado, com id, vaga_id, inicio e fim. Vaga compõe uma ou mais ocorrências; inicio deve ser anterior a fim. A candidatura abrange todas as ocorrências da vaga. O calendário é derivado de Voluntario → Candidatura → Vaga → OcorrenciaVaga, preservando o status da candidatura. Disponibilidade continua representando preferências do perfil, não compromissos assumidos.

Figura 7 – Diagrama de classes do domínio



Fonte: Elaborada pelos autores.

Cada classe é mapeada para uma tabela do MySQL pelo SQLAlchemy, e a especialização de Usuario é implementada por herança com tabelas associadas (*joined table inheritance*). Os atributos de tipo enumerado assumem valores fixos, armazenados como colunas do tipo ENUM no MySQL — por exemplo, StatusVaga (ABERTA, ENCERRADA

ou SUSPENSA) e StatusCandidatura (PENDENTE, ACEITA, RECUSADA ou CANCELADA).

2.6 VISÃO DE IMPLANTAÇÃO

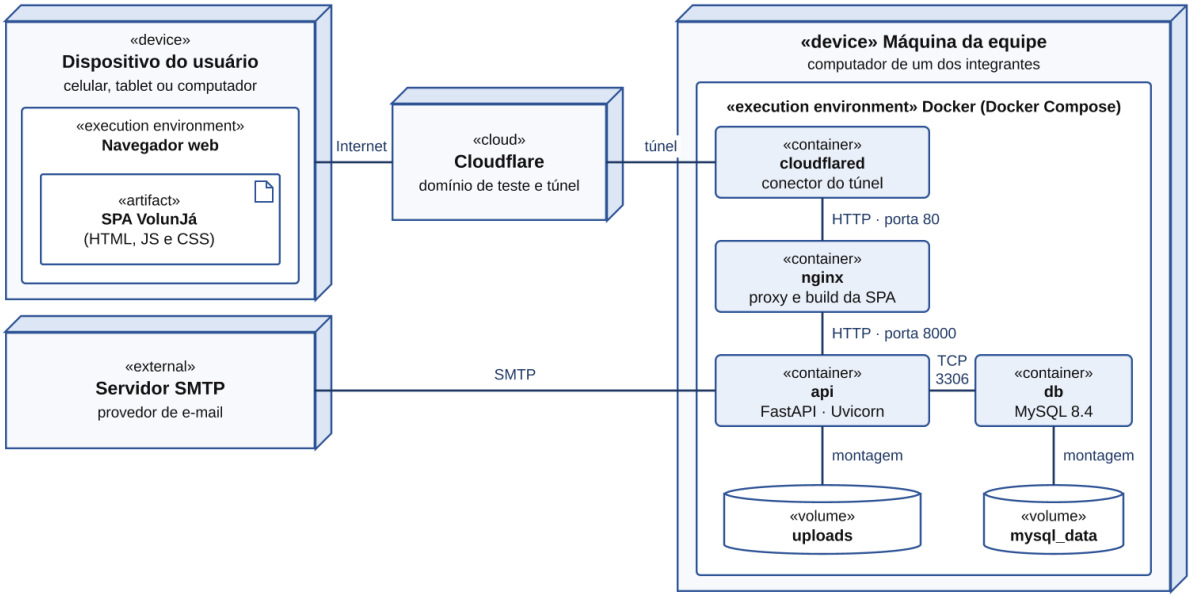
O VolunJá é implantado em contêineres Docker, orquestrados pelo Docker Compose, em uma máquina da própria equipe. Por se tratar de um produto mínimo viável (MVP) de uso acadêmico, com dezenas de usuários simultâneos nas validações e nas apresentações, um computador da equipe atende à carga prevista e dispensa custos de hospedagem. A aplicação é publicada na Internet por um túnel da Cloudflare (Cloudflare, 2026), em um domínio de teste, sem a necessidade de um servidor dedicado.

Além do cloudflared, que conecta a máquina ao túnel, a composição tem três contêineres: o **Nginx**, que entrega o *build* da SPA e encaminha as requisições /api à API; a **API**, em que o FastAPI é executado pelo servidor ASGI Uvicorn; e o banco de dados **MySQL 8.4 (LTS)**, acessível apenas pela rede interna do Docker. Os dados e os documentos das candidaturas ficam em volumes persistentes, e os e-mails são enviados por um provedor SMTP externo. O MySQL, definido no DV (Quadro 5), é um sistema gerenciador de banco de dados relacional maduro e gratuito, adequado a dados fortemente relacionados — vagas, candidaturas e documentos — que exigem integridade referencial, e possui suporte nativo no SQLAlchemy.

No desenvolvimento, cada integrante executa a mesma composição na própria máquina, e a integração contínua roda no GitHub Actions a cada *pull request*.

A Figura 8 apresenta o diagrama de implantação da aplicação.

Figura 8 – Diagrama de implantação do VolunJá



Fonte: Elaborada pelos autores.

A documentação do projeto, em MkDocs, continua publicada no GitHub Pages (DV, Quadro 5), de forma independente da aplicação.

2.7 RESTRIÇÕES ADICIONAIS

Além das metas e restrições do Quadro 4, o software está sujeito às seguintes restrições negociais:

a) o software é acessado diretamente pela Internet, por navegador, sem instalação; a consulta às vagas é pública (perfil Visitante), mas a candidatura, a publicação de vagas e a moderação exigem cadastro e *login* (DV, Tabela 4);
b) uma conta pertence a um único perfil, voluntário ou organização (DV, Tabela 4);
c) as vagas de uma organização só ficam visíveis após a homologação do cadastro pelo administrador (RF24); enquanto esse requisito, de prioridade Should, não for implementado, os cadastros são aprovados automaticamente;
d) o tratamento de dados pessoais — nome, CPF, contatos e documentos — segue a Lei Geral de Proteção de Dados Pessoais, LGPD (Brasil, 2018): coleta do mínimo necessário, consentimento no cadastro, acesso restrito às partes envolvidas em cada candidatura e exclusão da conta a pedido do titular;
e) nos testes, nas validações e nas apresentações são usados apenas dados fictícios, sem documentos ou dados pessoais reais;

f) o software é dimensionado, no MVP, para até 100 usuários autenticados simultaneamente, número compatível com o contexto acadêmico de validação;
g) por ser hospedado em uma máquina da equipe, sem custos de hospedagem, o sistema não tem garantia formal de disponibilidade contínua;
h) a interface é oferecida em português do Brasil, com datas no formato dd/mm/aaaa e horário de Brasília.

9) a candidatura deve obedecer às regras RN01 a RN05 do DV: bloquear conflitos com candidaturas ativas, informar a vaga conflitante e preservar a consistência da agenda em operações concorrentes (RF26 e RF27).

As características de qualidade relevantes para o software, segundo o modelo de qualidade da norma ISO/IEC 25010 (International Organization for Standardization, 2011), estão descritas no Quadro 8.

Quadro 8 – Características de qualidade relevantes para o software

Característica	Como é atendida	Justificativa
Usabilidade (inclui acessibilidade)	Protótipos no Figma, testes de usabilidade a cada incremento, diretrizes WCAG 2.1 no nível AA (W3C, 2018) e mensagens de erro claras.	Foco em UX do tema (DV, Quadro 1); RNF01 e RNF03; métrica M6.
Segurança	JWT com expiração, *hash* Argon2, autorização por perfil e por propriedade, validação com Pydantic, ORM contra injeção de SQL e limites no envio de arquivos.	RNF04 e RNF05; proteção de dados pessoais (Brasil, 2018).
Confiabilidade	Transações no banco de dados, dados em volumes persistentes, tratamento centralizado de erros e testes automatizados.	Evitar a perda de candidaturas e de documentos.
Eficiência de desempenho	Índices nos campos de filtro, paginação, e-mails enviados em segundo plano e arquivos estáticos com *cache* no Nginx.	RNF06 e meta MR03.
Manutenibilidade	Monólito modular em camadas, padrões do Apêndice A e cobertura de testes de, no mínimo, 80%.	RNF07 e prática de código coletivo (DV, seção 3.2).
Portabilidade	Contêineres Docker e SPA compatível com navegadores modernos em celular, tablet e desktop.	RNF02 e meta MR08.
Compatibilidade	API documentada em OpenAPI, o que permite integrar outros clientes no futuro, como um aplicativo móvel.	Evolução do produto.

Fonte: Elaborado pelos autores (2026).

Como perfis diferentes acessam a aplicação, a segurança merece atenção especial. O Quadro 9 apresenta as permissões de cada perfil, derivadas da Tabela 4 do DV.

Quadro 9 – Matriz de permissões por perfil

Funcionalidade	Visitante	Voluntário	Organização	Administrador
Buscar e visualizar vagas (RF12 a RF14)	Sim	Sim	Sim	Sim
Cadastrar-se (RF01 a RF03)	Sim	—	—	—
Manter o próprio perfil (RF06 e RF07)	—	Sim	Sim	—
Gerir vagas (RF08 a RF11)	—	—	Próprias	—
Candidatar-se com documentos (RF16 e RF17)	—	Sim	—	—
Acompanhar e cancelar candidaturas (RF18)	—	Próprias	—	—
Triar candidaturas (RF19)	—	—	Das próprias vagas	—
Acessar documentos de candidaturas (RNF04)	—	Próprios	Das próprias vagas	—
Visualizar contato após o aceite (RF21)	—	Próprias	Das próprias vagas	—
Emitir comprovante (RF22)	—	Próprio	—	—
Denunciar vaga ou usuário (RF25)	—	Sim	Sim	—
Homologar organizações e moderar (RF24 e RF25)	—	—	—	Sim
Manter usuários (DV, Tabela 4)	—	—	—	Sim
Consultar calendário (RF26)	—	Próprio	—	—

Fonte: Elaborado pelos autores (2026).

As permissões são aplicadas em duas etapas: (i) **autorização por perfil**, por dependências do FastAPI que verificam o perfil contido no *token* JWT antes de executar o *endpoint*; e (ii) **autorização por propriedade**, nos *services*, que verificam se o recurso pertence ao usuário autenticado, como a vaga que uma organização tenta editar. O administrador não acessa os documentos das candidaturas, pois não participa da relação entre voluntário e organização, em respeito ao princípio da necessidade da LGPD (Brasil, 2018). Além disso, o *token* expira em 60 minutos, as senhas têm no mínimo oito caracteres e as tentativas repetidas de *login* são limitadas no Nginx.

REFERÊNCIAS

ASSOCIAÇÃO BRASILEIRA DE NORMAS TÉCNICAS. **NBR 14724**: informação e documentação: trabalhos acadêmicos: apresentação. Rio de Janeiro: ABNT, 2011.

ASSOCIAÇÃO BRASILEIRA DE NORMAS TÉCNICAS. **NBR 6023**: informação e documentação: referências: elaboração. Rio de Janeiro: ABNT, 2018.

ASSOCIAÇÃO BRASILEIRA DE NORMAS TÉCNICAS. **NBR 10520**: informação e documentação: citações em documentos: apresentação. Rio de Janeiro: ABNT, 2023.

BASS, Len; CLEMENTS, Paul; KAZMAN, Rick. **Software architecture in practice**. 4. ed. Boston: Addison-Wesley, 2021.

BRASIL. **Lei nº 13.709, de 14 de agosto de 2018**. Lei Geral de Proteção de Dados Pessoais (LGPD). Brasília, DF: Presidência da República, 2018. Disponível em: https://www.planalto.gov.br/ccivil_03/_ato2015-2018/2018/lei/l13709.htm. Acesso em: 5 out. 2026.

CLOUDFLARE. **Cloudflare Tunnel**. [S. l.]: Cloudflare, 2026. Disponível em: https://developers.cloudflare.com/cloudflare-one/networks/connectors/cloudflare-tunnel/. Acesso em: 6 out. 2026.

CONVENTIONAL COMMITS. **Conventional Commits 1.0.0**. [S. l.], 2019. Disponível em: https://www.conventionalcommits.org/pt-br/v1.0.0/. Acesso em: 5 out. 2026.

DOCKER. **Docker documentation**. [S. l.]: Docker Inc., 2026. Disponível em: https://docs.docker.com/. Acesso em: 5 out. 2026.

FASTAPI. **FastAPI documentation**. [S. l.], 2026. Disponível em: https://fastapi.tiangolo.com/. Acesso em: 5 out. 2026.

FIELDING, Roy Thomas. **Architectural styles and the design of network-based software architectures**. 2000. Tese (Doutorado em Information and Computer Science) – University of California, Irvine, 2000.

FOWLER, Martin. **Patterns of enterprise application architecture**. Boston: Addison-Wesley, 2002.

GRUPO GOSLING. **VolunJá**: visão do produto e do projeto. Versão 0.3. Brasília, 2026. Documento da disciplina Métodos de Desenvolvimento de Software.

INTERNATIONAL ORGANIZATION FOR STANDARDIZATION. **ISO/IEC 25010**: systems and software engineering: systems and software quality requirements and evaluation (SQuaRE): system and software quality models. Geneva: ISO, 2011.

KRUCHTEN, Philippe B. The 4+1 view model of architecture. **IEEE Software**, Los Alamitos, v. 12, n. 6, p. 42-50, nov. 1995.

MYSQL. **MySQL 8.4 reference manual**. [S. l.]: Oracle, 2026. Disponível em: https://dev.mysql.com/doc/refman/8.4/en/. Acesso em: 5 out. 2026.

OBJECT MANAGEMENT GROUP. **OMG Unified Modeling Language (OMG UML)**: version 2.5.1. Needham: OMG, 2017. Disponível em: https://www.omg.org/spec/UML/2.5.1/. Acesso em: 5 out. 2026.

OPENAPI INITIATIVE. **OpenAPI specification**: version 3.1.0. [S. l.], 2021. Disponível em: https://spec.openapis.org/oas/v3.1.0. Acesso em: 5 out. 2026.

PRESSMAN, Roger S.; MAXIM, Bruce R. **Engenharia de software**: uma abordagem profissional. 8. ed. Porto Alegre: AMGH, 2016.

REACT. **React documentation**. [S. l.]: Meta Open Source, 2026. Disponível em: https://react.dev/. Acesso em: 5 out. 2026.

SOMMERVILLE, Ian. **Engenharia de software**. 10. ed. São Paulo: Pearson, 2018.

SQLALCHEMY. **SQLAlchemy 2.0 documentation**. [S. l.], 2026. Disponível em: https://docs.sqlalchemy.org/en/20/. Acesso em: 5 out. 2026.

VAN ROSSUM, Guido; WARSAW, Barry; COGHLAN, Alyssa. **PEP 8**: style guide for Python code. [S. l.]: Python Software Foundation, 2001. Disponível em: https://peps.python.org/pep-0008/. Acesso em: 5 out. 2026.

W3C. **Web Content Accessibility Guidelines (WCAG) 2.1**. [S. l.]: W3C, 2018. Disponível em: https://www.w3.org/TR/WCAG21/. Acesso em: 5 out. 2026.

APÊNDICE A – PADRÕES DE CODIFICAÇÃO E DE VERSIONAMENTO

Este apêndice detalha os padrões referidos na meta MR04 e no requisito RNF07 (DV, Tabela 6). O cumprimento dos padrões é verificado automaticamente na integração contínua e na revisão dos *pull requests*, como parte do conceito de pronto (DV, seção 3.3). O Quadro 10 resume as convenções de nomenclatura.

Quadro 10 – Convenções de nomenclatura

Elemento	Python (*back-end*)	JavaScript (*front-end*)
Arquivos	*snake_case* (models.py, vaga_utils.py)	*camelCase* (vagaService.js); componentes em *PascalCase* (VagaCard.jsx)
Classes e componentes	*PascalCase* (VagaService, VagaCreate)	*PascalCase* (VagaCard, FiltroVagas)
Funções e variáveis	*snake_case* (listar_vagas)	*camelCase* (listarVagas)
Constantes	*UPPER_SNAKE_CASE* (TAMANHO_MAXIMO_ARQUIVO)	*UPPER_SNAKE_CASE* (API_URL)
Hooks	—	Prefixo use (useAuth, useVagas)
Idioma dos nomes	Termos de domínio em português, sem acentos (vaga, candidatura); termos técnicos em inglês (*Service*, *Repository*, Schema)	Mesmo critério do *back-end*

Fonte: Elaborado pelos autores (2026).

A.1 *BACK-END* (PYTHON)

No *back-end*, adotam-se os seguintes padrões:

a) estilo PEP 8 (Van Rossum; Warsaw; Coghlan, 2001), verificado e formatado pelo Ruff, com linhas de até 88 caracteres;
b) anotações de tipo obrigatórias em funções públicas; toda entrada e saída da API é definida por *schemas* Pydantic;
c) cada módulo de domínio contém os arquivos router.py, schemas.py, service.py, repository.py e models.py, e as regras de negócio ficam apenas em service.py;
d) erros de negócio são representados por exceções próprias, convertidas em respostas HTTP por um tratador central — por exemplo, a exceção CandidaturaDuplicadaError;
e) testes com pytest na pasta tests, espelhando a estrutura dos módulos, com nomes no formato test_<comportamento_esperado>;
f) configurações e segredos ficam apenas em variáveis de ambiente: o arquivo .env não é versionado, e o arquivo .env.example documenta as variáveis necessárias.

A.2 *FRONT-END* (JAVASCRIPT E REACT)

No *front-end*, adotam-se os seguintes padrões:

a) ESLint, com as regras recomendadas e os *plugins* react-hooks e jsx-a11y, e Prettier para a formatação;
b) apenas componentes funcionais com *hooks*, com um componente por arquivo;
c) chamadas à API feitas somente pelos módulos da pasta *services*, nunca diretamente nos componentes;
d) estilos em CSS Modules (Componente.module.css), usando os *tokens* de *design* definidos a partir do protótipo;
e) acessibilidade: elementos HTML semânticos, rótulos em todos os campos, textos alternativos em imagens, foco visível e navegação por teclado;
f) testes com Vitest e React Testing Library no mesmo diretório do componente, como VagaCard.test.jsx.

A.3 API REST E BANCO DE DADOS

Na API e no banco de dados, adotam-se as seguintes convenções:

a) recursos nomeados no plural e em português (/vagas, /candidaturas), com o prefixo /api/v1;
b) métodos HTTP com a semântica padrão: GET consulta, POST cria, PUT substitui, PATCH altera parcialmente e DELETE remove;
c) JSON com chaves em *snake_case* e datas no padrão ISO 8601; listas paginadas com os campos itens, total, pagina e tamanho; erros no formato {"detail": "mensagem"};
d) tabelas em *snake_case* e no plural (vagas, candidaturas), com chave primária id, chaves estrangeiras no formato <entidade>_id e colunas de auditoria criado_em e atualizado_em;
e) toda alteração no esquema do banco de dados é feita por migração do Alembic, versionada no Git; o banco nunca é alterado manualmente.

A.4 VERSIONAMENTO

No versionamento do código, adotam-se as seguintes regras:

a) *branches*: main (versão estável), developer (integração) e *branches* de trabalho criadas a partir da developer — feature/<cenário>-<descrição> (por exemplo, feature/cn06-candidatura), fix/<descrição> e docs/<descrição>;
b) mensagens de *commit* no padrão Conventional Commits (Conventional Commits, 2019), no formato <tipo>(<escopo>): <descrição>, com os tipos feat, fix, docs, test, refactor, style e chore — por exemplo, feat(candidaturas): valida documentos exigidos;
c) toda alteração entra na developer por *pull request* revisado por, no mínimo, um integrante, com *lint* e testes aprovados;
d) a main recebe a developer ao fim de cada sprint, após a revisão com o Dono do Produto (P.O.).